This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Seeking Consistent Flat Minima for Better Domain Generalization via Refining Loss Landscapes

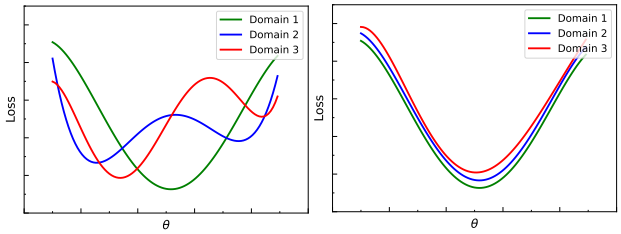
Aodi Li[1], Liansheng Zhuang[1,(✉)] Xiao Long[1], Minghong Yao[1], Shafei Wang[2]
[1]University of Science and Technology of China, Hefei 230026, China
[2]Peng Cheng Laboratory, Shenzhen 518000, China
aodili@mail.ustc.edu.cn, lszhuang@ustc.edu.cn

Abstract

Domain generalization aims to learn a model from multiple training domains and generalize it to unseen test domains. Recent theory has shown that seeking the deep models, whose parameters lie in the flat minima of the loss landscape, can significantly reduce the out-of-domain generalization error. However, existing methods often neglect the consistency of loss landscapes in different domains, resulting in models that are not simultaneously in the optimal flat minima in all domains, which limits their generalization ability. To address this issue, this paper proposes an iterative Self-Feedback Training (SFT) framework to seek consistent flat minima that are shared across different domains by progressively refining loss landscapes during training. It alternatively generates a feedback signal by measuring the inconsistency of loss landscapes in different domains and refines these loss landscapes for greater consistency using this feedback signal. Benefiting from the consistency of the flat minima within these refined loss landscapes, our SFT helps achieve better out-of-domain generalization. Extensive experiments on DomainBed demonstrate superior performances of SFT when compared to state-of-the-art sharpness-aware methods and other prevalent DG baselines. On average across five DG benchmarks, SFT surpasses the sharpness-aware minimization by 2.6% with ResNet-50 and 1.5% with ViT-B/16, respectively.

1. Introduction

The task of Domain Generalization (DG) is to learn a model from multiple training domains so that it can generalize well to unseen test domains [59]. Though modern deep learning has achieved remarkable success in many areas [29, 43, 58, 69], it assumes that training and test data are independent and identically distributed (IID). This assumption is often violated in real applications, which significantly degrades the performance of deep models [59]. To address this problem, abundant domain generalization (DG) algorithms have been developed from different perspectives, e.g., invariant or causal representation learning [1, 17, 66], disentangled representation learning [3, 47], distributionally robust optimization [51], meta-learning [5, 13, 37], data augmentation [57, 64], etc. Nonetheless, it is still an open problem for the communities of modern machine learning.



(a) Inconsistent loss landscapes (b) Loss landscapes with consistency

Figure 1. **Loss Landscapes without and with consistency.** The left subplot (a) illustrates the inconsistency of loss landscapes across different domains, which arise due to domain shifts. This paper proposes refining these landscapes to achieve improved consistency, as demonstrated in the right subplot (b).

Recently, the loss landscape [39], which refers to the shape of a loss function within the parameter space, provides a unified perspective for understanding various DG algorithms. Essentially, the training process of deep models is to seek the minimum point within the training loss landscape. From the view of loss landscapes [39], the ultimate goal of various DG methods is to construct a suitable loss landscape so that the loss minimum point sought on training domains is exactly the loss minimum point on unseen test domains. In fact, significant studies have revealed the connection between the geometry of the loss landscape (particularly the flatness of minima [20]) and generalization from both theoretical and empirical perspectives [14, 16, 27, 30]. This connection has shown potential in enabling novel methods to model training that yield better generalization [16]. For example, by penalizing the sharpness of the loss landscape, numerous methods [11, 16, 45] have exhibited superior generalization on the DomainBed

benchmark [19] and many other datasets [2, 53]. However, in the DG scenario, the domain shifts contribute to the drastic discrepancy of loss landscapes [9], and thus loss landscapes in test domains may not exhibit similar geometries as in training domains. Consequently, the flat minima sought on the training domains do not have a low loss value in the unseen test domains, meaning that the learned models may have inferior performance in the test domains. Therefore, maintaining consistency of the loss landscapes between the training and unseen test domains is crucial for better generalization, as it enables us to seek the consistent flat minima that perform well across all domains.

To address this issue, this paper proposes refining the loss landscapes to maintain their consistency in different domains. However, there are several primary challenges to be addressed. Firstly, in the DG setting, the inaccessibility of the test domain restricts us to using limited training domains for refining loss landscapes. Then, how to ensure that the loss landscape consistency achieved on training domains can transfer to unseen test domains becomes a challenging endeavor. Secondly, refining the loss landscapes usually means high complexity. Typically, it involves the selection of different model architectures [4, 18, 41, 63] or the artificial design of novel loss functions [1, 33, 65]. Architecture selection suffers from substantial computational overhead [62], while artificially designing an effective loss function for DG is also exceedingly intractable. Therefore, finding efficient approaches to refine various training loss landscapes is far from straightforward.

Inspired by studies on the self-refinement of large-scale language models [42, 44], this paper introduces an iterative two-phase framework called *Self-Feedback Training* (SFT) to progressively refine loss landscapes and find consistent flat minima across different domains. Specifically, it alternatively generates a feedback signal by measuring the inconsistency of loss landscapes in different domains in the *feedback* phase, and then refines these loss landscapes for greater consistency using this feedback signal in the *refinement* phase. To ensure the consistency of loss landscapes between the training and unseen test domains, this paper introduces a training-domain split scheme in the feedback phase. In each iteration, the SFT quantifies the inconsistency by comparing the loss landscapes of the trained and held-out domains. Theoretical analysis (in the supplementary file) shows that in this way, the sharpness of the test loss can be constrained by that of the training domains, which also means the transferability of the loss landscapes consistency. Furthermore, to enable low-complexity refinement of loss landscapes, a landscape refiner is introduced in the refinement phase. Instead of altering the model architecture or devising novel loss functions, the refiner continuously modifies the geometries of the loss landscapes by generating dynamical soft labels as training progresses. Compared to existing methods [4, 41], our landscape refining method not only substantially reduces the computational overhead, but also overcomes the over-confident issue brought by traditional one-hot labels [46]. To our best knowledge, this is the first work to refine the loss landscapes from a label-based perspective. Benefiting from the consistency of the flat minima within these refined loss landscapes, our SFT helps achieve better out-of-domain generalization. Extensive experiments on a synthetic dataset and five popular DG benchmarks demonstrate that SFT achieves superior performance compared to popular sharpness-aware methods [16, 61, 67, 68, 72] and other prevalent DG methods.

In summary, our key contributions are as follows:

- A novel self-feedback training framework, which could refine loss landscapes and find consistent flat minima across different domains, is proposed to obtain models with better domain generalization.
- A landscape refiner is introduced to facilitate the efficient refinement of loss landscapes. To our best knowledge, it is the first work to refine the loss geometry from the perspective of labels.
- Extensive experiments on the synthetic dataset and five popular DG benchmarks demonstrate the superiority of SFT over popular sharpness-aware methods and other prevalent DG methods.

2. Related Work

In this section, we review related areas of research: studies on loss landscapes for model generalization and domain generalization.

Loss landscapes for model generalization. The idea of searching for "flat" minima of loss landscapes can be traced back to Hochreiter and Schmidhuber [21], who proposed penalizing the sharpness of minima as a form of regularization, related to the Minimum Description Length (MDL) principle. Over the past few decades, numerous studies [14, 27, 30] have linked the flatness of loss landscapes to the generalization ability of deep models. As a result, many optimization methods have been developed to find flatter minima, such as Entropy-SGD [11] and the diffusion approach [45]. More recently, two methods, Stochastic Weight Averaging (SWA) [23] and Sharpness-Aware Minimization (SAM) [16], have gained significant attention for their effectiveness and scalability. SWA finds flatter minima by averaging model parameters across the optimization trajectory, while SAM seeks parameters within regions of the loss landscape that exhibit uniformly low loss values. Jean Kaddour *et al.* [28] provided a detailed comparison of these methods, showing that SAM tends to lead models to wider basins, while SWA merely helps locate the center of one certain basin. Several efficient variants of SAM [12, 24, 26] have also been developed for real-world applications. In this paper, we focus on addressing the inconsistency of loss

landscapes when SAM is applied in DG scenarios.

Loss landscapes for domain generalization. Recently, optimization techniques aiming at finding flat minima in loss landscapes have also been applied to improve the out-of-domain generalization of deep models [73]. Junbum Cha *et al.* [9] introduced the flatness to the domain generalization and proposed a novel Stochastic Weight Averaging Densely (SWAD), which is an extension of SWA with a dense and overfit-aware stochastic weight sampling strategy. Zhang *et al.* [68] pointed out that SAM-based methods are limited to zeroth-order flatness, which may be insufficient to discriminate minima with low generalization error. Thus, they developed a novel method named GAM, which targets first-order flatness to enhance generalization. Moreover, FAD [67] optimizes both zeroth- and first-order flatness simultaneously for domain generalization. However, these methods primarily focus on seeking better flat minima for DG, ignoring the influence of landscape discrepancy across multiple domains. In contrast, Giambattista Parascandolo *et al.* [48] considered the domain discrepancy via formalizing a notion of consistency for minima of loss surfaces, and then proposed an algorithm (ANDMask) to alter the optimization trajectory through masking inconsistent gradient components among training domains. Different from all of the above methods that consider loss landscapes to be static, we attempt to refine the loss surfaces dynamically for consistency and finally seek consistent flat minima across various domains.

3. Methodology

3.1. Problem Formulation and Preliminaries

Let us begin with a formal description of *domain* and *domain generalization (DG)*. Let $\mathcal{X}$ and $\mathcal{Y}$ denote the input sample space and the category space, respectively. In the DG problem, we use data sampled from p training distributions $\{\mathcal{D}_d\}_{d=1}^{p}$ and q test distributions $\{\mathcal{D}_d\}_{d=p+1}^{p+q}$, each of which defines on the joint space $\mathcal{X} \times \mathcal{Y}$. $D_d = \{(\boldsymbol{x}_i^{(d)}, y_i^{(d)})\}_{i=1}^{n_d}$ denotes the dataset sampled from the d-th distribution $\mathcal{D}_d$, which is referred to as the d-th *domain*. $(\boldsymbol{x}_i^{(d)}, y_i^{(d)}) \in \mathcal{X} \times \mathcal{Y}$ denotes the i-th sample from the domain $\mathcal{D}_d$ and n_d denotes the number of samples in the d-th domain. Notably, each domain contains different domain statistics but shares the same category space. *Domain generalization* aims to train a model $\boldsymbol{f}_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}$ on the p training domains so that it can generalize well to the q novel target domains. Please note that the q test domains are inaccessible during training, which differs from the problem of domain adaptation [60].

In this paper, we consider the model mentioned above to be a parametric deep neural network $\boldsymbol{f}_{\boldsymbol{\theta}}$, with $\boldsymbol{\theta}$ denoting the parameters of the network. Standard ERM training of networks usually utilizes the cross entropy (CE) as the loss function:

$$\mathcal{L}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta}) = -\sum_{d=1}^{p}\sum_{i=1}^{n_d} \mathbf{y}_i^{(d)\mathbf{T}} \log \boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^{(d)}), \tag{1}$$

where $\mathcal{L}_{D_{tr}}^{\mathrm{CE}}$ represents the cross-entropy loss computed on the training set, which is an aggregation of samples from all the training domains. In this paper, we use the superscript on $\mathcal{L}$ to indicate the type of loss function (e.g., cross entropy), while the subscript denotes the dataset on which the loss is calculated. $\boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^{(d)})$ is a vector with N dimensions (where N is the number of classes), and the j-th entry of the vector represents the predictive probability that $\boldsymbol{x}_i^{(d)}$ belongs to the j-th class. $\mathbf{y}_i^{(d)}$ denotes the one-hot encoded label of $y_i^{(d)}$, whose superscript "$\mathbf{T}$" indicates the matrix transpose operation.

However, optimizing the training loss value only (e.g., ERM) can easily lead to suboptimal model quality [16]. Thus, SAM [16] tries to simultaneously minimize loss value and loss sharpness. It defines the sharpness by measuring how quickly the training loss can be increased by adding a small perturbation $\boldsymbol{\epsilon}$ (with norm less than ρ) to the current parameter $\boldsymbol{\theta}$. To avoid sharpness being dependent on the perturbation $\boldsymbol{\epsilon}$, the worst case is considered when evaluating sharpness:

$$\mathcal{L}_{D_{tr}}^{\mathrm{CES}}(\boldsymbol{\theta}) = \max_{||\boldsymbol{\epsilon}|| \le \rho} \mathcal{L}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta} + \boldsymbol{\epsilon}) - \mathcal{L}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta}). \tag{2}$$

Here, we use $\mathcal{L}_{D_{tr}}^{\mathrm{CES}}$ to denote the sharpness measure for $\mathcal{L}_{D_{tr}}^{\mathrm{CE}}$. Since the perturbation strength ρ is small enough, the above optimization could be solved approximately via Taylor expansion, leading to the optimal perturbation $\bar{\boldsymbol{\epsilon}}_{D_{tr}}^{\mathrm{CE}}$ for $\mathcal{L}_{D_{tr}}^{\mathrm{CE}}$ as:

$$\bar{\boldsymbol{\epsilon}}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta}) = \rho \frac{\nabla_{\boldsymbol{\theta}} \mathcal{L}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta})}{||\nabla_{\boldsymbol{\theta}} \mathcal{L}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta})||}. \tag{3}$$

Then, the final loss function used in SAM combines both the loss value and the loss sharpness, and is given by:

$$\mathcal{L}_{D_{tr}}^{\mathrm{SAM}}(\boldsymbol{\theta}) = \mathcal{L}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta} + \bar{\boldsymbol{\epsilon}}_{D_{tr}}^{\mathrm{CE}}). \tag{4}$$

During training with (stochastic) gradient descent, the contribution of $\nabla_{\boldsymbol{\theta}} \bar{\boldsymbol{\epsilon}}_{D_{tr}}^{\mathrm{CE}}(\boldsymbol{\theta})$ could be neglected due to small perturbation strength ρ.

3.2. Self-Feedback Training

To address the issue of landscape inconsistency when applying SAM-based methods to DG scenarios, we introduce an iterative two-phase framework called *Self-Feedback Training* (SFT) to progressively refine loss landscapes and find consistent flat minima across different domains. Specifically, it alternatively generates a feedback signal by measuring the inconsistency of loss landscapes in different domains in the *feedback* phase, and then refines these loss

Algorithm 1: SFT for Domain Generalization.

Input: Training domains D_{tr}, learning rate η and other hyperparameters.

Output: Model parameter $\boldsymbol{\theta}$.

Init: Parameters of the model $\boldsymbol{f}_{\boldsymbol{\theta}}$ and refiner $\boldsymbol{g}_{\boldsymbol{\phi}}$;

while *not converge* **do**

  The Feedback Phase:

  Randomly select two training domains D_d and $D_{d'}$.

  Update the model parameter $\boldsymbol{\theta}$ on the domain D_d: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}_{D_d}^{\mathrm{SAM-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi})$.

  Calculate the feedback signal $|\mathcal{L}_{D_d}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) - \mathcal{L}_{D_{d'}}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi})|$ using both domains.

  The Refinement Phase:

  Calculate the total loss for refinement, $\mathcal{L}_{D_{tr}}^{\mathrm{Refine}}(\boldsymbol{\theta}, \boldsymbol{\phi})$, by leveraging the above feedback signal.

  Update the refiner parameter: $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \eta \nabla_{\boldsymbol{\phi}} \mathcal{L}_{D_{tr}}^{\mathrm{Refine}}(\boldsymbol{\theta}, \boldsymbol{\phi})$.

end

landscapes for greater consistency using this feedback signal in the *refinement* phase. In the following, this paper provides a detailed description of this framework.

3.2.1. The Feedback Phase

In this phase, SFT mainly detects inconsistency between various loss landscapes and generates a feedback signal.

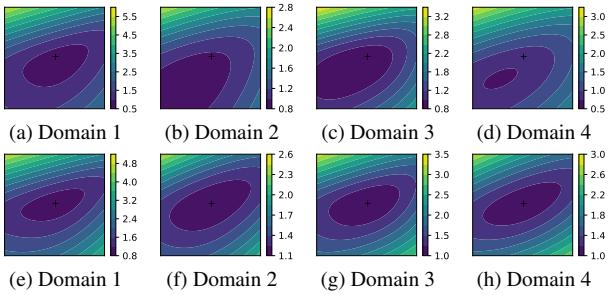
Let's start by analyzing the key elements necessary for obtaining a feedback signal. First, due to the absence of test domains, we are limited to assessing the landscape inconsistency within the training domains. Making full use of the training domains to simulate domain shifts that may occur in unseen domains will help improve the model's generalization ability to those unknown domains. Second, the training loss should depend on some learnable parameters, which enables us to optimize them for landscape refinement in the next phase. Finally, we need to determine which measurement to use when assessing the landscape inconsistency.

Considering the above analysis, we choose to implement a training-domain split scheme, where at each iteration, two training domains, D_d and $D_{d'}$, are randomly selected. D_d is used to train the model, while $D_{d'}$ is held out to evaluate landscape inconsistency in comparison to D_d. During training on the domain D_d, we employ the training loss that depends on the parameters of a landscape refiner $\boldsymbol{g}_{\boldsymbol{\phi}}$. This refiner enables the efficient refinement of loss landscapes via generating soft labels for each sample:

$$\tilde{\mathbf{y}}_i^{(d)} = \boldsymbol{g}_{\boldsymbol{\phi}}(\boldsymbol{x}_i^{(d)}). \tag{5}$$

Then, the loss function for sharpness-aware minimization can be expressed as:

$$\mathcal{L}_{D_d}^{\mathrm{SAM-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \mathcal{L}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta} + \bar{\boldsymbol{\epsilon}}_{D_d}^{\mathrm{CE-SL}}, \boldsymbol{\phi}). \tag{6}$$



(a) Domain 1 (b) Domain 2 (c) Domain 3 (d) Domain 4

(e) Domain 1 (f) Domain 2 (g) Domain 3 (h) Domain 4

Figure 2. **2D visualization of loss surfaces at each domain with/without landscape refinement.** The first row shows the inconsistency of loss surfaces using one-hot labels (without refinement); the second row shows the improved landscape consistency using soft labels generated by the landscape refiner. The final well-trained model is marked by "+".

Here, "SL" represents the utilization of soft labels. $\mathcal{L}_{D_d}^{\mathrm{CE-SL}}$ and $\bar{\boldsymbol{\epsilon}}_{D_d}^{\mathrm{CE-SL}}$ are defined as:

$$\mathcal{L}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) = -\sum_{i=1}^{n_d} \tilde{\mathbf{y}}_i^{(d)\mathbf{T}} \log \boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x}_i^{(d)}), \tag{7}$$

$$\bar{\boldsymbol{\epsilon}}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \rho \frac{\nabla_{\boldsymbol{\theta}} \mathcal{L}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi})}{||\nabla_{\boldsymbol{\theta}} \mathcal{L}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi})||}. \tag{8}$$

For evaluating landscape inconsistency, one direct approach is to look at the difference in loss values across domains. However, the loss value only captures zero-order information. Considering that loss sharpness is closely related to the Hessian matrix and reflects second-order loss information, we instead use the difference in loss sharpness, $|\mathcal{L}_{D_d}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) - \mathcal{L}_{D_{d'}}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi})|$, to quantify the landscape consistency, which serves as a feedback signal to the landscape refiner. Here, $\mathcal{L}_{D_d}^{\mathrm{CES-SL}}$ is define as:

$$\mathcal{L}_{D_d}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \mathcal{L}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta} + \bar{\boldsymbol{\epsilon}}_{D_d}^{\mathrm{CE-SL}}, \boldsymbol{\phi}) - \mathcal{L}_{D_d}^{\mathrm{CE-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}). \tag{9}$$

Similarly, $\mathcal{L}_{D_{d'}}^{\mathrm{CES-SL}}$ can be defined. Please note that the landscape refiner parameter $\boldsymbol{\phi}$ will be optimized in the refinement phase using the feedback signal and could be treated as constant in this phase.

3.2.2. The Refinement Phase

After obtaining the feedback signal, this phase will use it to optimize the landscape refiner to enhance the consistency of different landscapes.

As mentioned earlier, the landscape refiner refines the loss landscapes by generating soft labels for subsequent training. There are several important considerations about these labels. First, it is essential to maintain the correctness of the labels as much as possible, as this directly impacts the effectiveness of training. Second, the soft labels help improve the consistency of the refined loss landscapes, which is central to addressing the landscape inconsistency

problem discussed in this paper. Moreover, there are two potential directions to refining the loss landscapes for consistency: making them both sharper or making them both flatter. Given the relationship between flatness and generalization, the soft labels need to be designed to encourage the loss landscapes to become flatter.

To ensure label correctness, we could use the standard cross-entropy loss to force the generated soft labels to approximate one-hot labels. However, this approach carries the risk of overfitting to the one-hot labels, leading to overconfidence and limiting the model's generalization ability. One technique that attempts to mitigate this issue is label smoothing [46], but it merely introduces noise to the one-hot labels, making them less informative and unable to capture the underlying relationships between categories and different input samples. This paper focuses on a more fundamental problem: minimizing the "distance" between the refiner's output and the true label space for each category. For instance, if the first category is the true label for the sample $\boldsymbol{x}_i$ (i.e., $y_i = 1$), the corresponding label space is defined as:

$$C_1 = \{(q_1, \ldots, q_N) \mid \forall k \neq 1 : q_1 \geq \alpha q_k, \sum_{k=1}^{N} q_k = 1\}, \tag{10}$$

where $\alpha \geq 1$ is a hyperparameter that represents the minimum ratio between q_1 and q_k. Similar to the concept of "distance" in Euclidean space, we define the "distance" between the output $\tilde{\mathbf{y}} = \boldsymbol{g}_{\boldsymbol{\phi}}(\boldsymbol{x})$ and the label space C_1 as:

$$\min_{\mathbf{y}} \mathrm{KL}(\mathbf{y} \,||\, \tilde{\mathbf{y}}) \quad \text{subject to} \quad \mathbf{y} \in C_1, \tag{11}$$

where $\mathrm{KL}(\cdot\|\cdot)$ denotes the Kullback-Leibler (KL) divergence. We propose an efficient algorithm (Algorithm 2) to find the optimal solution $\mathbf{y}^\star$ for this problem, which converges in N steps (N is the number of classes) and is computationally more efficient than using common convex programming tools. For the condition that $y_i \neq 1$, we can also get the optimal solution via category swapping before and after applying Algorithm 2. Once having $\mathbf{y}^\star$, the task becomes minimizing the KL divergence between $\mathbf{y}^\star$ and $\tilde{\mathbf{y}}$: $\mathrm{KL}(\mathbf{y}^\star \,||\, \tilde{\mathbf{y}})$. Since $\mathbf{y}^\star$ depends on $\boldsymbol{\phi}$ and Algorithm 2 may involve non-differentiable operations, we adopt a widely-used alternative iterative strategy: we treat $\mathbf{y}^\star$ as constant during each update step for $\boldsymbol{\phi}$, and after updating $\boldsymbol{\phi}$, we recalculate $\mathbf{y}^\star$ using Algorithm 2. Minimizing the KL divergence then leads to an objective similar to cross entropy:

$$\mathcal{L}_{D_{tr}}^{\mathrm{PCE}}(\boldsymbol{\phi}) = -\sum_{d=1}^{p}\sum_{i=1}^{n_d} \mathbf{y}_i^{\star(d)\mathbf{T}} \log \boldsymbol{g}_{\boldsymbol{\phi}}(\boldsymbol{x}_i^{(d)}), \tag{12}$$

which we refer to as projection cross-entropy (PCE) loss. Interestingly, we find that label smoothing is just a special case of our method, applicable only to a small subset of samples. More detailed derivations and further discussion are available in the **supplementary file**.

Algorithm 2: KL Divergence Minimization

Input: The hyperparameter α, $\tilde{\mathbf{y}} = (p_1, \ldots, p_N)$.
Output: The optimal solution $\mathbf{y}^\star = (q_1, \ldots, q_N)$.

Initialization: $A \leftarrow \emptyset$, $B \leftarrow \{j | \alpha p_j > p_1\}$, $t \leftarrow 1$.
Sort the elements of B in descending order: $p_{j_1} \geq \cdots \geq p_{j_{|B|}}$.
Update: $A \leftarrow A \cup \{j_1\}$, $t \leftarrow t + 1$.
while $t \leq |B|$ **do**
  if $\left(p_1^\alpha \left(\prod_{j \in A} \alpha p_j\right)\right)^{\frac{1}{|A|+\alpha}} < \alpha p_{j_t}$ **then**
    Update: $A \leftarrow A \cup \{j_t\}$, $t \leftarrow t + 1$.
  else
    Break
  end
end
Calculate q_1: $q_1 = \left(p_1^\alpha \left(\prod_{j \in A} \alpha p_j\right)\right)^{\frac{1}{|A|+\alpha}}$.
for $i \in \{2, \ldots, N\}$ **do**
  if $i \in A$ **then**
    $q_i = q_1 / \alpha$.
  else
    $q_i = p_i$.
  end
end
Normalize $q_1, \ldots, q_N$ such that $\sum_i q_i = 1$.

To encourage the loss landscapes to become flatter while pursuing consistency, we add the sharpness term to the previously obtained feedback signal (the sharpness difference) as an extra penalty. Then, the final loss function is defined as:

$$\begin{aligned}\mathcal{L}_{D_{tr}}^{\mathrm{Refine}}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \mathcal{L}_{D_{tr}}^{\mathrm{PCE}}(\boldsymbol{\phi}) + \lambda_1 \mathcal{L}_{D_d}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) \\ + \lambda_2 |\mathcal{L}_{D_d}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi}) - \mathcal{L}_{D_{d'}}^{\mathrm{CES-SL}}(\boldsymbol{\theta}, \boldsymbol{\phi})|.\end{aligned} \tag{13}$$

Here, the hyperparameters λ_1 and λ_2 control the relative importance of each term in the loss function. By using this loss function to update the landscape refiner $\boldsymbol{g}_{\boldsymbol{\phi}}$, the generated soft labels could help adjust the loss landscapes, making them more consistent and flatter over time.

As the iterative process of feedback and refinement continues, the model benefits from these refined landscapes, guiding it toward more stable and consistent optimization paths. Ultimately, the model can seek the consistent flat minima across domains, which helps achieve better out-of-domain generalization.

4. Experiments

In this section, we begin by designing a toy experiment to visually demonstrate the landscape consistency achieved through self-feedback training, and explore how the landscape consistency impacts DG performance. A thorough

theoretical analysis is provided in the **supplementary file**. After that, we perform extensive experiments on real-world datasets to validate the effectiveness of the SFT method.

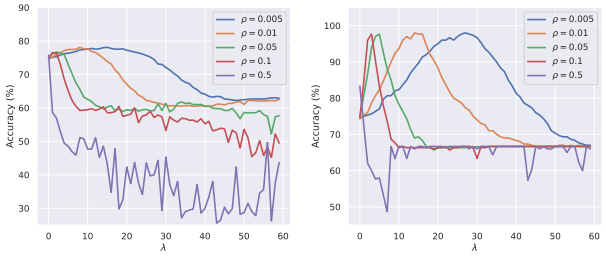
4.1. Experiments on Toy Dataset

Data generation. In our toy experiments, we use a hierarchical Gaussian model. Specifically, we assume that the data from the i-th class in the j-th domain follows a two-dimensional Gaussian distribution, i.e., $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_{ij}, \boldsymbol{\Sigma}_{ij})$, where $\boldsymbol{\mu}_{ij} \in \mathbb{R}^2$ and $\boldsymbol{\Sigma}_{ij} \in \mathbb{R}^{2\times 2}$ are the mean vector and covariance matrix for the j-th domain in the i-th class, respectively. The mean vector $\boldsymbol{\mu}_{ij}$ is sampled from another Gaussian distribution: $p(\boldsymbol{\mu}_{ij}) = \mathcal{N}(\boldsymbol{\mu}_{ij}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$, where $\boldsymbol{\mu}_i$ and $\boldsymbol{\Sigma}_i$ represent the mean and covariance for the i-th class across domains. In our setup, we generate a dataset with three classes and four domains. The first three domains are used for training, while the fourth domain serves as the test set. Further details can be found in the **supplementary file**. A linear classifier is employed in these toy experiments.

Loss landscape visualization. To investigate the consistency of the landscape across the four domains after self-feedback training, we visualize the 2D loss surfaces in Fig.2. We first choose three model weights $(\theta_1, \theta_2, \theta_3)$,[1] and use them to derive two axes, e_1 and e_2, through the Gram–Schmidt process. Then, we compute loss values by varying the coefficients β_1 and β_2 in the linear combination $\theta_1 + \beta_1 e_1 + \beta_2 e_2$, where $\beta_1, \beta_2 \in [-2, 2]$ in our experiments. From Fig.2, we observe that the loss surfaces using soft labels generated by the landscape refiner exhibit greater consistency across domains compared to those obtained using one-hot labels. Besides, we find that the final model sought by SFT, marked by a "+", indeed locates the flat minima within the more consistent loss surfaces. These results demonstrate the effectiveness of SFT.

Influence of the landscape consistency on DG performances. In this experiment, we gradually increase the values of λ_1, λ_2 to strengthen the promotion of consistent flatness, while tracking the corresponding changes in DG performance. For simplicity, we set $\lambda_1 = \lambda_2 = \lambda$ throughout the experiment. As shown in Fig.3, if the perturbation strength ρ is enough small, the out-of-domain accuracy initially increases and then decreases as λ increases (see Fig.3b). This phenomenon can be explained as a trade-off between maintaining label correctness and enhancing the landscape consistency: Initially, with a small value of λ, the soft labels remain accurate, and increasing λ helps improve out-of-domain generalization by encouraging landscape consistency. However, when λ becomes too large, it compromises the correctness of the soft labels, leading to a sharp drop in accuracy for both the training (see Fig.3a) and test domains. Besides, as the perturbation strength ρ increases, the accuracy peak on the test domain shifts to the left, which is because a larger sharpness value will be obtained with larger ρ, alleviating the need for excessively large λ to promote consistency. Finally, when ρ reaches 0.5, accuracies on the training and test domains deteriorate rapidly, similar to the behavior of the SAM [16].

[1]Here, θ_1 represents the parameters of the well-trained model, while θ_2 and θ_3 correspond to two randomly initialized models.



(a) Training Domains (b) Test Domain

Figure 3. **Performances of classification with the varied hyperparameter λ.** The left (a) and right subplots (b) show the performance on the training and test domains, respectively.

4.2. Experiments on Real Dataset

4.2.1. Experimental setting

Dataset and protocol. We evaluate our method on five widely-used and challenging datasets for domain generalization, including PACS [36] (4 domains, 7 classes, and 9, 991 examples), VLCS [15] (4 domains, 5 classes, and 10, 729 examples), OfficeHome [56] (4 domains, 65 classes, and 15, 588 images), TerraIncognita [6] (4 domains, 10 classes, and 24, 788 examples) and DomainNet [49] (6 domains, 345 classes, and 586, 575 examples). Our experimental environment is built based on the well-known DomainBed benchmark [19]. Following the training and evaluation protocol of DomainBed, we select one domain for testing and the remaining domains for training every time, and 20% samples of training domains are held out for validation and model selection. Model selection is carried out based on the training-domain validation set.

Implementation details. To illustrate the broad applicability of the proposed method, we conduct experiments based on small models (ImageNet pre-trained ResNet-50) and large-scale pre-trained models (ViT-B/16 and ViT-L/14 for CLIP [50]). Unless otherwise specified, the landscape refiner shares the same architecture with the model $\boldsymbol{f_\theta}$ in this paper. Through random hyperparameter search, we determine the batch size, learning rates, and other hyperparameters. The detailed search spaces for each are provided in the **supplementary file**. The Adam [32] optimizer is utilized in all of our experiments.

4.2.2. Main Results

Experiments with the ResNet-50 backbone. We begin by comparing SFT with conventional DG methods. As shown in Table 1, SFT outperforms ERM [55] across all five

Table 1. **Comparison with popular DG methods with ResNet-50 pre-trained on ImageNet.** The table reports the average out-of-domain accuracy on five DG datasets. Each out-of-domain performance is an average of three different runs with distinct train-validation splits. We highlight the best results in **bold** and underline the second best results. Results marked by † and ‡ are cited from Gulrajani and Lopez-Paz [19] and Cha *et al.* [9], respectively.

Algorithms	VLCS	PACS	OfficeHome	TerraIncognita	DomainNet	Avg.
ERM† [55]	77.5±0.4	85.5±0.2	66.5±0.3	46.1±1.8	40.9±0.1	63.3
IRM† [1]	78.5±0.5	83.5±0.8	64.3±2.2	47.6±0.8	33.9±2.8	61.6
GroupDRO† [51]	76.7±0.6	84.4±0.8	66.0±0.7	43.2±1.1	33.3±0.2	60.7
Mixup† [64]	77.4±0.6	84.6±0.6	68.1±0.3	47.9±0.8	39.2±0.1	63.4
MLDG† [37]	77.2±0.4	84.9±1.0	66.8±0.6	47.7±0.9	41.2±0.1	63.6
CORAL† [54]	78.8±0.6	86.2±0.3	68.7±0.3	47.6±1.0	41.5±0.1	64.6
MMD† [38]	77.5±0.9	84.6±0.5	66.3±0.1	42.2±1.6	23.4±9.5	58.8
DANN† [17]	78.6±0.4	83.6±0.4	65.9±0.6	46.7±0.5	38.3±0.1	62.6
CDANN† [40]	77.5±0.1	82.6±0.9	65.8±1.3	45.8±1.6	38.3±0.3	62.0
MTL† [7]	77.2±0.4	84.6±0.5	66.4±0.5	45.6±1.2	40.6±0.1	62.9
SagNet† [47]	77.8±0.5	86.3±0.2	68.1±0.1	48.6±1.0	40.3±0.1	64.2
ARM† [65]	77.6±0.3	85.1±0.4	64.8±0.3	45.5±0.3	35.5±0.2	61.7
VREx† [33]	78.3±0.2	84.9±0.6	66.4±0.6	46.4±0.6	33.6±2.9	61.9
RSC† [22]	77.1±0.5	85.2±0.9	65.5±0.9	46.6±1.0	38.9±0.5	62.7
Mixstyle‡ [71]	77.9±0.5	85.2±0.3	60.4±0.3	44.0±0.7	34.0±0.1	60.3
AndMask [48]	78.1±0.9	84.4±0.9	65.6±0.4	44.6±0.3	37.2±0.6	62.0
Fish [52]	77.8±0.3	85.5±0.3	68.6±0.4	45.1±1.3	42.7±0.2	63.9
SelfReg† [31]	77.8±0.9	85.6±0.4	67.9±0.7	47.0±0.3	42.8±0.0	64.2
mDSDI [8]	79.0±0.3	86.2±0.2	69.2±0.4	48.1±1.4	42.8±0.1	65.1
MIRO [10]	79.0±0.0	85.4±0.4	70.5±0.4	50.4±1.1	44.3±0.2	65.9
SAM‡ [16]	79.4±0.1	85.8±0.2	69.6±0.1	43.3±0.7	44.3±0.0	64.5
SFT (Ours)	**79.8**±0.1	**88.3**±0.3	**70.9**±0.1	**50.7**±0.4	**46.0**±0.0	**67.1**

Table 2. **DG performances on large-scale Vision Transformers.** Out-of-domain accuracies of two backbones (ViT-B/16 and ViT-L/14) are shown below. The presence or absence of "*" indicates whether full fine-tuning or visual prompt tuning is employed, respectively.

Backbone	**Algorithms**	VLCS	PACS	OfficeHome	TerraIncognita	DomainNet	Avg.
ViT-B/16	ERM* [55]	81.4	92.9	78.9	53.6	56.1	72.6
	MIRO* [10]	82.2	95.6	82.5	54.3	54.0	73.7
	ERM [55]	80.9	96.6	84.1	55.5	59.2	75.3
	IRM [1]	81.9	96.4	83.1	50.9	59.1	74.3
	DANN [17]	81.7	95.5	82.7	52.0	58.6	74.1
	CDANN [40]	81.9	96.0	82.3	54.9	58.4	74.7
	CORAL [54]	82.5	95.4	83.3	52.0	59.5	74.5
	MMD [38]	81.9	95.1	83.7	56.9	59.9	75.5
	IIB [35]	82.5	96.0	83.9	58.0	58.6	75.8
	SAM [16]	83.5	96.1	85.7	56.6	59.8	76.3
	SFT (Ours)	**84.1**	**96.8**	**86.5**	**61.2**	**60.5**	**77.8**
ViT-L/14	ERM [55]	82.9	**98.8**	90.2	61.3	65.4	79.7
	SAM [16]	84.2	98.7	90.8	62.8	65.2	80.3
	SFT (Ours)	**84.4**	98.6	**91.3**	**65.2**	**66.5**	**81.2**

datasets, showing an average improvement of 3.8%. Furthermore, SFT consistently outperforms other conventional DG methods, including invariant or causal representation learning methods (IRM [1], VREx [33], CORAL [54], MMD [38], DANN [17], CDANN [40]), disentangled representation learning methods (SagNet [47] and mDSDI [8]), distributionally robust optimization methods (GroupDRO [51]), meta-learning methods (MLDG [37], ARM [65], and Fish [52]), data or feature augmentation methods (Mixup [64] and Mixstyle [71]), and other methods (RSC [22], AndMask [48], MTL [7], and SelfReg [31]). As a strong baseline, SAM achieves an average improvement of +1.2% (63.3% → 64.5%) over ERM and outperforms most other baselines, except for mDSDI [8] and MIRO [10]. By refining loss landscapes to improve consistency, SFT provides an additional average gain of +2.6% (64.5% → 67.1%) over SAM. Remarkably, our SFT even outperforms the robust MIRO [10] baseline by 1.2%.

Experiments with Large-scale Vision Transformers (ViTs). To demonstrate the versatility of SFT across different architectures, we also conduct experiments using large-scale ViTs. Given the efficiency of Visual Prompt Tuning (VPT) [25]—which requires tuning only 1% of the parameters—and its strong performance in domain general-

Table 3. **Comparisons of SFT and popular sharpness-aware methods.** This table presents average DG performances on five datasets, and the mean and standard deviation for each dataset.

Algorithms	VLCS	PACS	OfficeHome	TerraIncognita	DomainNet	Avg.
SAM [16]	79.4±0.1	85.8±0.2	69.6±0.1	43.3±0.7	44.3±0.0	64.5
GAM [68]	78.5±0.4	86.1±0.6	68.2±1.0	45.2±0.6	43.8±0.1	64.4
GSAM [72]	79.1±0.2	85.9±0.1	69.3±0.0	47.0±0.8	44.6±0.2	65.1
FAD [67]	78.9±0.8	88.2±0.5	69.2±0.5	45.7±1.0	44.4±0.1	65.3
SAGM [61]	**80.0**±0.3	86.6±0.2	70.1±0.2	48.8±0.9	45.0±0.2	66.1
SFT (Ours)	79.8±0.1	**88.3**±0.3	**70.9**±0.1	**50.7**±0.4	**46.0**±0.0	**67.1**

ization, as shown in Table 2 and previous works [34, 70], we adopt VPT in our ViT experiments. From Table 2, we can see that SFT achieves superior performance over conventional DG methods and the vanilla SAM (+1.5% average improvement) when using ViT-B/16. When using a larger pre-trained ViT-L/14, the out-of-domain accuracies further improve, achieving 79.7% for ERM, 80.3% for SAM, and 81.2% for SFT. The results highlight that SFT can be easily integrated with popular tuning methods like VPT, and there is potential for further enhancement with advanced tuning techniques in future work.

Comparison with popular sharpness-aware methods. We also perform a more fine-grained comparison of SFT with popular SAM variants designed to seek better flat minima. As shown in Table 3, SFT generally outperforms these methods across all five datasets, with average improvements of 2.7%, 2.0%, 1.8% and 1.0% over GAM [68], GSAM [72], FAD [67] and SAGM [61], respectively. These experimental results further highlight the importance of the consistency of flat minima. Since SFT currently only applies the basic SAM algorithm, there is potential for further improvements by incorporating more advanced SAM variants, such as SAGM [61], which remains a focus for our future research.

4.3. Sharpness Analysis

We also perform a quantitative analysis of the landscape consistency. As mentioned earlier, SFT quantifies landscape consistency through domain-specific sharpness differences. In Figure 4, we find that SFT reduces domain-specific sharpness values and their differences compared to SAM, while simultaneously attaining higher classification accuracy. These empirical results confirm that SFT effectively improves the loss landscape consistency across domains, while also demonstrating the positive impact of landscape consistency on the DG performance.

4.4. Ablation Study

Ablation studies on the key components of the SFT framework are summarized in Table 4. The upper part of the table shows experimental results for ERM and SAM with one-hot and smoothed labels, while the lower part presents results using soft labels generated by the landscape refiner. The results in the upper part confirm the effectiveness of SAM

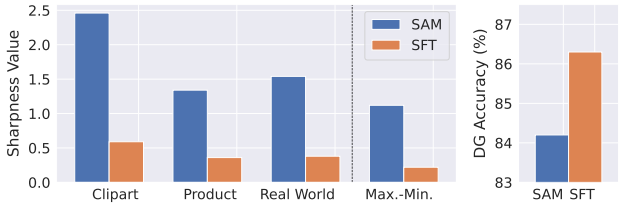


Figure 4. **Comparison of model sharpness and DG accuracy across different training strategies on the OfficeHome dataset.**

(where $\rho \neq 0$) and label smoothing. When comparing the six experiments in the lower part, we observe that incorporating all components of the SFT framework to enhance landscape consistency across domains yields the best performance, achieving an average accuracy of 86.5%. Furthermore, removing any component from the SFT framework results in a performance drop, which underscores the importance of each element within the framework. Finally, the comparison between the upper and lower sections demonstrates that the consistency-guided soft labels in our SFT framework offer a clear advantage over traditional one-hot or smoothed labels, significantly improving model generalization.

Table 4. **Ablative experiments on the OfficeHome dataset.** The "Label" column specifies the type of labels used, such as one-hot labels, smoothed labels or labels generated by the landscape refiner. The "PCE" indicates whether the PCE loss is utilized in the Exp.5-10. If not, we use the standard cross entropy with label smoothing instead.

Exp.	ρ	Label	PCE	λ_1	λ_2	Art	Clipart	Product	Real World	Avg.
1	0.0	One-hot	–	–	–	83.2	73.8	90.1	89.1	84.0
2	0.0	smoothed	–	–	–	83.6	74.1	90.0	89.6	84.3
3	0.3	One-hot	–	–	–	83.8	76.6	90.1	90.3	85.2
4	0.3	Smoothed	–	–	–	84.2	76.4	90.6	90.5	85.4
5	0.0	Generated	w/o	0.0	0.0	83.9	74.9	89.9	90.0	84.7
6	0.3	Generated	w/o	0.0	0.0	84.0	75.9	90.1	90.1	85.0
7	0.3	Generated	w/o	0.5	0.0	84.8	76.8	90.3	90.6	85.6
8	0.3	Generated	w/o	0.0	0.7	84.9	77.3	90.6	90.5	85.8
9	0.3	Generated	w/o	0.5	0.7	85.2	77.2	90.8	90.7	86.0
10	0.3	Generated	w/	0.5	0.7	**86.3**	**77.7**	**91.2**	**90.9**	**86.5**

5. Conclusion

This paper has proposed an iterative two-phase Self-Feedback Training (SFT) framework, aiming at addressing the issue of landscape inconsistency brought by domain shifts. It alternatively generates a feedback signal by measuring the inconsistency of loss landscapes in different domains during the feedback phase, and then refines these loss landscapes for greater consistency using this feedback signal in the refinement phase. Benefiting from the consistency of the sought flat minima, SFT demonstrates superior performances over sharpness-aware methods and prevalent DG methods in diverse experimental settings. We hope that this study will inspire more research on advanced DG methods from the view of loss landscapes. Further theoretical analysis and technical improvement will be our future work.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant No.U20B2070 and No.61976199.

References

[1] Martin Arjovsky, Léon Bottou, Ishaan Gulrajani, and David Lopez-Paz. Invariant risk minimization. *arXiv preprint arXiv:1907.02893*, 2019. 1, 2, 7

[2] Dara Bahri, Hossein Mobahi, and Yi Tay. Sharpness-aware minimization improves language model generalization. *arXiv preprint arXiv:2110.08529*, 2021. 2

[3] Haoyue Bai, Rui Sun, Lanqing Hong, Fengwei Zhou, Nanyang Ye, Han-Jia Ye, S-H Gary Chan, and Zhenguo Li. Decaug: Out-of-distribution generalization via decomposed feature representation and semantic augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 6705–6713, 2021. 1

[4] Haoyue Bai, Fengwei Zhou, Lanqing Hong, Nanyang Ye, S-H Gary Chan, and Zhenguo Li. Nas-ood: Neural architecture search for out-of-distribution generalization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8320–8329, 2021. 2

[5] Yogesh Balaji, Swami Sankaranarayanan, and Rama Chellappa. Metareg: Towards domain generalization using meta-regularization. *Advances in neural information processing systems*, 31, 2018. 1

[6] Sara Beery, Grant Van Horn, and Pietro Perona. Recognition in terra incognita. In *Proceedings of the European conference on computer vision (ECCV)*, pages 456–473, 2018. 6

[7] Gilles Blanchard, Aniket Anand Deshmukh, Ürun Dogan, Gyemin Lee, and Clayton Scott. Domain generalization by marginal transfer learning. *The Journal of Machine Learning Research*, 22(1):46–100, 2021. 7

[8] Manh-Ha Bui, Toan Tran, Anh Tran, and Dinh Phung. Exploiting domain-specific features to enhance domain generalization. *Advances in Neural Information Processing Systems*, 34:21189–21201, 2021. 7

[9] Junbum Cha, Sanghyuk Chun, Kyungjae Lee, Han-Cheol Cho, Seunghyun Park, Yunsung Lee, and Sungrae Park. Swad: Domain generalization by seeking flat minima. *Advances in Neural Information Processing Systems*, 34: 22405–22418, 2021. 2, 3, 7

[10] Junbum Cha, Kyungjae Lee, Sungrae Park, and Sanghyuk Chun. Domain generalization by mutual-information regularization with pre-trained models. *arXiv preprint arXiv:2203.10789*, 2022. 7

[11] Pratik Chaudhari, Anna Choromanska, Stefano Soatto, Yann LeCun, Carlo Baldassi, Christian Borgs, Jennifer Chayes, Levent Sagun, and Riccardo Zecchina. Entropy-sgd: Biasing gradient descent into wide valleys. *Journal of Statistical Mechanics: Theory and Experiment*, 2019(12):124018, 2019. 1, 2

[12] Jiawei Du, Daquan Zhou, Jiashi Feng, Vincent Tan, and Joey Tianyi Zhou. Sharpness-aware training for free. *Advances in Neural Information Processing Systems*, 35: 23439–23451, 2022. 2

[13] Yingjun Du, Jun Xu, Huan Xiong, Qiang Qiu, Xiantong Zhen, Cees GM Snoek, and Ling Shao. Learning to learn with variational information bottleneck for domain generalization. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part X 16*, pages 200–216. Springer, 2020. 1

[14] Gintare Karolina Dziugaite and Daniel M Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *arXiv preprint arXiv:1703.11008*, 2017. 1, 2

[15] Chen Fang, Ye Xu, and Daniel N Rockmore. Unbiased metric learning: On the utilization of multiple datasets and web images for softening bias. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1657–1664, 2013. 6

[16] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. *arXiv preprint arXiv:2010.01412*, 2020. 1, 2, 3, 6, 7, 8

[17] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. Domain-adversarial training of neural networks. *The journal of machine learning research*, 17(1):2096–2030, 2016. 1, 7

[18] Soumya Suvra Ghosal, Yifei Ming, and Yixuan Li. Are vision transformers robust to spurious correlations? *arXiv preprint arXiv:2203.09125*, 2022. 2

[19] Ishaan Gulrajani and David Lopez-Paz. In search of lost domain generalization. *arXiv preprint arXiv:2007.01434*, 2020. 2, 6, 7

[20] Sepp Hochreiter and Jürgen Schmidhuber. Simplifying neural nets by discovering flat minima. *Advances in neural information processing systems*, 7, 1994. 1

[21] Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural computation*, 9(1):1–42, 1997. 2

[22] Zeyi Huang, Haohan Wang, Eric P Xing, and Dong Huang. Self-challenging improves cross-domain generalization. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II 16*, pages 124–140. Springer, 2020. 7

[23] Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. *arXiv preprint arXiv:1803.05407*, 2018. 2

[24] Jie Ji, Gen Li, Jingjing Fu, Fatemeh Afghah, Linke Guo, Xiaoyong Yuan, and Xiaolong Ma. A single-step, sharpness-aware minimization is all you need to achieve efficient and accurate sparse training. *Advances in Neural Information Processing Systems*, 37:44269–44290, 2024. 2

[25] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. *arXiv preprint arXiv:2203.12119*, 2022. 7

[26] Weisen Jiang, Hansi Yang, Yu Zhang, and James Kwok. An adaptive policy to employ sharpness-aware minimization. *arXiv preprint arXiv:2304.14647*, 2023. 2

[27] Yiding Jiang, Behnam Neyshabur, Hossein Mobahi, Dilip Krishnan, and Samy Bengio. Fantastic generalization measures and where to find them. *arXiv preprint arXiv:1912.02178*, 2019. 1, 2

[28] Jean Kaddour, Linqing Liu, Ricardo Silva, and Matt J Kusner. When do flat minima optimizers work? *Advances in Neural Information Processing Systems*, 35:16577–16595, 2022. 2

[29] Uday Kamath, John Liu, and James Whitaker. *Deep learning for NLP and speech recognition*. Springer, 2019. 1

[30] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016. 1, 2

[31] Daehee Kim, Youngjun Yoo, Seunghyun Park, Jinkyu Kim, and Jaekoo Lee. Selfreg: Self-supervised contrastive regularization for domain generalization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9619–9628, 2021. 7

[32] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 6

[33] David Krueger, Ethan Caballero, Joern-Henrik Jacobsen, Amy Zhang, Jonathan Binas, Dinghuai Zhang, Remi Le Priol, and Aaron Courville. Out-of-distribution generalization via risk extrapolation (rex). In *International Conference on Machine Learning*, pages 5815–5826. PMLR, 2021. 2, 7

[34] Aodi Li, Liansheng Zhuang, Shuo Fan, and Shafei Wang. Learning common and specific visual prompts for domain generalization. In *Proceedings of the Asian Conference on Computer Vision*, pages 4260–4275, 2022. 8

[35] Bo Li, Yifei Shen, Yezhen Wang, Wenzhen Zhu, Dongsheng Li, Kurt Keutzer, and Han Zhao. Invariant information bottleneck for domain generalization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 7399–7407, 2022. 7

[36] Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy M Hospedales. Deeper, broader and artier domain generalization. In *Proceedings of the IEEE international conference on computer vision*, pages 5542–5550, 2017. 6

[37] Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy Hospedales. Learning to generalize: Meta-learning for domain generalization. In *Proceedings of the AAAI conference on artificial intelligence*, 2018. 1, 7

[38] Haoliang Li, Sinno Jialin Pan, Shiqi Wang, and Alex C Kot. Domain generalization with adversarial feature learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5400–5409, 2018. 7

[39] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, 31, 2018. 1

[40] Ya Li, Mingming Gong, Xinmei Tian, Tongliang Liu, and Dacheng Tao. Domain generalization via conditional invariant representations. In *Proceedings of the AAAI conference on artificial intelligence*, 2018. 7

[41] Yanxi Li, Zhaohui Yang, Yunhe Wang, and Chang Xu. Adapting neural architectures between domains. *Advances in Neural Information Processing Systems*, 33:789–798, 2020. 2

[42] Xun Liang, Shichao Song, Zifan Zheng, Hanyu Wang, Qingchen Yu, Xunkai Li, Rong-Hua Li, Feiyu Xiong, and Zhiyu Li. Internal consistency and self-feedback in large language models: A survey. *arXiv preprint arXiv:2407.14507*, 2024. 2

[43] Marc Moreno Lopez and Jugal Kalita. Deep learning applied to nlp. *arXiv preprint arXiv:1703.03091*, 2017. 1

[44] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2024. 2

[45] Hossein Mobahi. Training recurrent neural networks by diffusion. *arXiv preprint arXiv:1601.04114*, 2016. 1, 2

[46] Rafael Müller, Simon Kornblith, and Geoffrey E Hinton. When does label smoothing help? *Advances in neural information processing systems*, 32, 2019. 2, 5

[47] Hyeonseob Nam, HyunJae Lee, Jongchan Park, Wonjun Yoon, and Donggeun Yoo. Reducing domain gap via style-agnostic networks. *arXiv preprint arXiv:1910.11645*, 2(7):8, 2019. 1, 7

[48] Giambattista Parascandolo, Alexander Neitz, Antonio Orvieto, Luigi Gresele, and Bernhard Schölkopf. Learning explanations that are hard to vary. *arXiv preprint arXiv:2009.00329*, 2020. 3, 7

[49] Xingchao Peng, Qinxun Bai, Xide Xia, Zijun Huang, Kate Saenko, and Bo Wang. Moment matching for multi-source domain adaptation. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1406–1415, 2019. 6

[50] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pages 8748–8763. PMLR, 2021. 6

[51] Shiori Sagawa, Pang Wei Koh, Tatsunori B Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *arXiv preprint arXiv:1911.08731*, 2019. 1, 7

[52] Yuge Shi, Jeffrey Seely, Philip HS Torr, N Siddharth, Awni Hannun, Nicolas Usunier, and Gabriel Synnaeve. Gradient matching for domain generalization. *arXiv preprint arXiv:2104.09937*, 2021. 7

[53] Hye-jin Shim, Jee-weon Jung, and Tomi Kinnunen. Multi-dataset co-training with sharpness-aware optimization for audio anti-spoofing. *arXiv preprint arXiv:2305.19953*, 2023. 2

[54] Baochen Sun and Kate Saenko. Deep coral: Correlation alignment for deep domain adaptation. In *European conference on computer vision*, pages 443–450. Springer, 2016. 7

[55] Vladimir Vapnik. Statistical learning theory. *(No Title)*, 1998. 6, 7

[56] Hemanth Venkateswara, Jose Eusebio, Shayok Chakraborty, and Sethuraman Panchanathan. Deep hashing network for unsupervised domain adaptation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5018–5027, 2017. 6

[57] Riccardo Volpi, Hongseok Namkoong, Ozan Sener, John C Duchi, Vittorio Murino, and Silvio Savarese. Generalizing to unseen domains via adversarial data augmentation. *Advances in neural information processing systems*, 31, 2018. 1

[58] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, and Eftychios Protopapadakis. Deep learning for computer vision: A brief review. *Computational intelligence and neuroscience*, 2018, 2018. 1

[59] Jindong Wang, Cuiling Lan, Chang Liu, Yidong Ouyang, Tao Qin, Wang Lu, Yiqiang Chen, Wenjun Zeng, and Philip Yu. Generalizing to unseen domains: A survey on domain generalization. *IEEE Transactions on Knowledge and Data Engineering*, 2022. 1

[60] Mei Wang and Weihong Deng. Deep visual domain adaptation: A survey. *Neurocomputing*, 312:135–153, 2018. 3

[61] Pengfei Wang, Zhaoxiang Zhang, Zhen Lei, and Lei Zhang. Sharpness-aware gradient matching for domain generalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3769–3778, 2023. 2, 8

[62] Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*, 2023. 2

[63] Chongzhi Zhang, Mingyuan Zhang, Shanghang Zhang, Daisheng Jin, Qiang Zhou, Zhongang Cai, Haiyu Zhao, Xianglong Liu, and Ziwei Liu. Delving deep into the generalization of vision transformers under distribution shifts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7277–7286, 2022. 2

[64] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. *arXiv preprint arXiv:1710.09412*, 2017. 1, 7

[65] Marvin Zhang, Henrik Marklund, Abhishek Gupta, Sergey Levine, and Chelsea Finn. Adaptive risk minimization: A meta-learning approach for tackling group shift. *arXiv preprint arXiv:2007.02931*, 8:9, 2020. 2, 7

[66] Xingxuan Zhang, Peng Cui, Renzhe Xu, Linjun Zhou, Yue He, and Zheyan Shen. Deep stable learning for out-of-distribution generalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5372–5382, 2021. 1

[67] Xingxuan Zhang, Renzhe Xu, Han Yu, Yancheng Dong, Pengfei Tian, and Peng Cui. Flatness-aware minimization for domain generalization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 5189–5202, 2023. 2, 3, 8

[68] Xingxuan Zhang, Renzhe Xu, Han Yu, Hao Zou, and Peng Cui. Gradient norm aware minimization seeks first-order flatness and improves generalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20247–20257, 2023. 2, 3, 8

[69] Zixing Zhang, Jürgen Geiger, Jouni Pohjalainen, Amr El-Desoky Mousa, Wenyu Jin, and Björn Schuller. Deep learning for environmentally robust speech recognition: An overview of recent developments. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 9(5):1–28, 2018. 1

[70] Zangwei Zheng, Xiangyu Yue, Kai Wang, and Yang You. Prompt vision transformer for domain generalization. *arXiv preprint arXiv:2208.08914*, 2022. 8

[71] Kaiyang Zhou, Yongxin Yang, Yu Qiao, and Tao Xiang. Domain generalization with mixstyle. *arXiv preprint arXiv:2104.02008*, 2021. 7

[72] Juntang Zhuang, Boqing Gong, Liangzhe Yuan, Yin Cui, Hartwig Adam, Nicha Dvornek, Sekhar Tatikonda, James Duncan, and Ting Liu. Surrogate gap minimization improves sharpness-aware training. *arXiv preprint arXiv:2203.08065*, 2022. 2, 8

[73] Yingtian Zou, Kenji Kawaguchi, Yingnan Liu, Jiashuo Liu, Mong-Li Lee, and Wynne Hsu. Towards robust out-of-distribution generalization bounds via sharpness. *arXiv preprint arXiv:2403.06392*, 2024. 3